CodeMind

Study guide

Everything we built, in plain words:
how it works, what every term means, and what is new.

How to use this guide

- Read parts 1 to 3 first. They tell the story: the problem, what a user does, and how the pieces fit together.
- Parts 4 and 5 are the technical heart: embeddings, search, the prompt, and every feature, step by step.
- Part 9 lists exactly what is new. Part 11 has the questions a teacher is likely to ask, with answers.
- Part 12 is a glossary. Every difficult word in this guide is explained there in simple language.
- You do not need to memorise code. Memorise the flows (the numbered boxes) and the numbers in part 10.

> **A note on honesty**
> Say only what is true. Some things are built and tested (streaming, compare, retrieval lab, explain, PDF report). Some are only set up and still need an account or a key (Sweep.dev and Qodo). Part 10 lists these clearly, so you are never caught out.

Project: CodeMind, an AI codebase engineer. Built on the open-source project git-inspect by Parth Pankaj Singh (MIT licence). Always give that credit.

Contents

1. The idea in one minute 4
- The problem 4
- What CodeMind does 4
- Why local-first matters 4
- Three ideas to remember 4

2. What a user does: the journey 5
- The screens, one by one 5

3. The big picture 7
- Where things live in the code 7

4. How the AI part works 8
- 4.1 What is an LLM? 8
- 4.2 Why a model alone is not enough: RAG 8
- 4.3 Embeddings: turning meaning into numbers 8
- 4.4 Where the vectors are stored 9
- 4.5 Chunking: cutting code into the right-sized pieces 9
- 4.6 Indexing: what happens when you connect a repository 10
- 4.7 Hybrid search: meaning plus exact words 10
- 4.8 Building the prompt and the guardrails 11
- 4.9 Generating and streaming the answer 11

5. The features, one by one 13
- 5.1 Ask AI with sources 13
- 5.2 Auto-scale: a small model first, a big one only if you agree 13
- 5.3 Streaming answers (new) 13
- 5.4 Chat history and follow-ups (new) 13
- 5.5 Compare models (new) 14
- 5.6 Retrieval lab (new) 14
- 5.7 Explain this file (new) 14
- 5.8 Export report (new) 15
- 5.9 Breaking-change detection 15
- 5.10 GitHub webhook: automatic analysis (new) 15
- 5.11 Metrics: every model call is measured 15

6. The database: what is stored 17

7. The frontend, explained 18
- 7.1 The tools 18
- 7.2 Fluid design: sizes that flow 18
- 7.3 Explaining things inside the screens 18
- 7.4 How the browser handles a streamed answer 19

8. Quality, DevOps and security 20
- 8.1 Testing 20
- 8.2 DevOps: automation around the code 20
- 8.3 Security and privacy 20

9. What is new, and what was there before 22
- Layer 0: the starting point (not our work) 22
- Layer 1: built before this week 22
- Layer 2: the engine work for the second evaluation 22
- Layer 3: the screens and features for the second evaluation 23

10. Numbers, limits and future scope 24
Numbers worth remembering (all measured) 24
Honest limits (say these before you are asked) 24
Future scope (each builds on something that already exists) 24
11. Presenting it tomorrow 26
Before you start (checklist) 26
A five-minute demo 26
Questions a teacher may ask, with answers 26
12. Glossary: every word in simple language 28

1. The idea in one minute

The problem

When a developer joins a project, or opens an old one, the first days go into reading code. Where is the login? What does this file do? What changed last week? In a big project nobody remembers everything.

A normal chatbot does not fix this. It has never seen your code, so it guesses. Pasting your code into a website also means sending private code to a company.

What CodeMind does

- You connect a GitHub repository. CodeMind reads it and builds a searchable index.
- You ask questions in plain English. CodeMind finds the relevant code, answers using only that code, and shows you the files it used as proof.
- It watches new commits and warns you about changes that might break things.
- All the AI runs on your own computer by default, so your code never has to leave it.

> **The one-sentence pitch**
> CodeMind is an AI assistant that has read your repository, answers questions about it with proof, and does not have to send your code to the cloud.

Why local-first matters

- Privacy: the code stays on your machine. This is the main selling point.
- Cost: no monthly bill and no API key needed.
- Works offline once the models are downloaded.
- The trade-off: a small local model is less clever than a big cloud model. We handle that with an optional, approved 'scale up' to a cloud model (part 5).

Three ideas to remember

1. RAG (Retrieval-Augmented Generation): look the answer up first, then write it. The model reads the right code before it answers, so it does not have to guess.
2. Embeddings: a way to turn the meaning of text into numbers, so a computer can search by meaning and not only by exact words.
3. Small private model by default, bigger cloud model only if you approve. You choose the balance between privacy and power.

2. What a user does: the journey

Follow one person from the first click to a finished report. Each box is one step.

1. **Connect a repository**
   Paste a GitHub link (and a token if the repository is private). CodeMind first shows which files it will index, then shows live progress: the stage, how many files are done, the file it is on, and timings.

2. **CodeMind reads the repository (indexing)**
   It downloads the code, picks the important files, writes a short AI summary of them, splits every file into function-sized pieces, and turns each piece into numbers called an embedding. Everything is saved in the database. Part 4 explains this slowly.

3. **Look at the dashboard**
   The dashboard shows how many files were indexed, which languages the code uses, whether Docker, CI and tests exist, the recent commits, and any commit flagged as a breaking change.

4. **Ask a question**
   Type something like 'How does login work?'. The sources appear first (file names, line numbers and a match score). Then the answer writes itself word by word, like ChatGPT. You can press Stop at any time.

5. **Ask a follow-up**
   Type 'and where is it used?'. CodeMind remembers your last 3 questions, so 'it' makes sense. The conversation is saved in your browser, so it is still there after a refresh.

6. **Dig deeper**
   Explain this file: a list of the functions plus a plain-English explanation. Compare models: ask one question to the local model, Gemini and Groq side by side. Retrieval lab: measure how good the search is. Semantic search: find code by meaning.

7. **Share and automate**
   Export a PDF report. When someone pushes code to GitHub, a webhook tells CodeMind and it analyses the new commits by itself. CI/CD checks every change before it is accepted.

The screens, one by one

Screen	What it is for	Status
Connect repository	Add a GitHub repository. Preview the files first, then watch live indexing progress.	Before
Dashboard	Overview numbers, the Ask box, languages and DevOps detection, commits, breaking-change monitor, Export report button.	Updated
Ask AI	Ask questions, follow-ups, and keep a list of saved answers.	Updated

Screen	What it is for	Status
Compare Models	The same question answered by two to four models side by side, with speed and size.	NEW
Semantic Search	Search the code by meaning instead of exact words.	Before
Retrieval Lab	Test how well the search finds the right files, with scores and charts.	NEW
Repository Files	Browse indexed files. Explain this file opens a side panel.	Updated
AI Metrics	Speed, tokens and errors for every model call, local or cloud.	Before
System Status	Is Ollama running? Are the models, the database and Qdrant healthy?	Before

3. The big picture

CodeMind has five layers. A request goes down through them and the answer comes back up.

1. Browser	What you see. Next.js and React pages, Tailwind styling. It shows the answer as it streams in, and keeps your chat in the browser.
↓	
2. Server	Next.js API routes and tRPC. They check the input, check you may see the project, run the search, call the model and stream the result back.
↓	
3. AI brain	Ollama runs two local models on your computer: qwen2.5-coder (writes answers) and nomic-embed-text (makes embeddings). Gemini and Groq are optional cloud models.
↓	
4. Memory	PostgreSQL with the pgvector add-on stores projects, files, code chunks, their vectors, commits, saved answers and metrics. Qdrant is installed for the future.
↓	
5. Outside world	GitHub: the source of the code and the commits, and the sender of webhooks when someone pushes.

Where things live in the code

Folder	What is inside
`src/app`	The pages and the API routes (ask, explain, report, webhook, tRPC).
`src/components`	Reusable screen pieces: answer panel, explainer box, info tooltip, buttons.
`src/server`	Server logic: the ask pipeline, explain, report data, database access rules.
`src/services/rag`	Search and answering: chunking, retrieval, hybrid search, prompts, evaluation.
`src/services/llm`	One class per model provider (Ollama, Gemini, Groq) behind one common shape.
`src/lib`	Helpers used by both the browser and the server.
`prisma`	The database schema (the list of tables).
`tests`	227 unit tests and 8 integration tests.
`scripts`	Command-line tools: npm run eval, npm run index:local.
`.github`	CI/CD workflows, the AI pull-request review, Qodo, issue template for Sweep.
`monitoring`	Prometheus and Grafana settings for live graphs.

> **Why this design is good to mention**
> Every model sits behind one common interface (LLMProvider). Adding a new provider means writing one small class. The rest of the app does not change.

4. How the AI part works

This is the part teachers ask about most. Read it slowly. Every new word is explained the first time it appears, and again in the glossary.

4.1 What is an LLM?

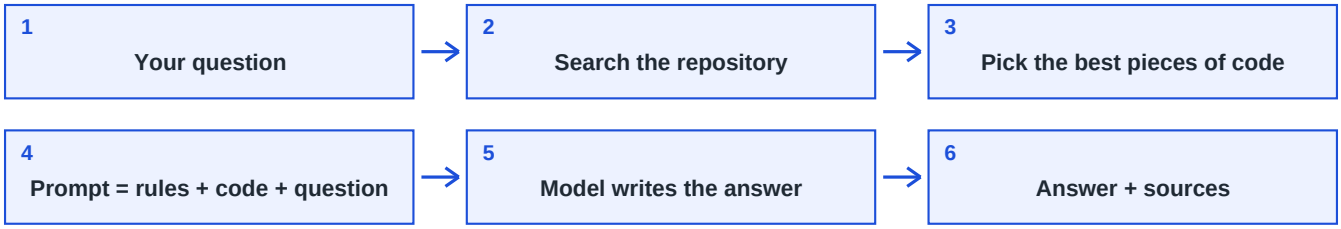
An LLM (Large Language Model) is a program trained on huge amounts of text. It learned to continue text: given some words, it predicts the next word, then the next, until it has written an answer. Our local model is qwen2.5-coder with 3 billion parameters, which is small, fast and trained on a lot of code. A 'parameter' is one learned number inside the model. More parameters usually means smarter but slower and bigger.

Word	Plain meaning
Token	A piece of a word. About 4 characters of English. 'chunking' is about 2 tokens. Models read and write tokens, not letters.
Context window	How much text the model can look at in one go: your question, the code we give it, and its own answer together. Ours is 3,072 tokens for the local model, roughly 2,000 words. That is small, which is why we must choose the code carefully.
Temperature	How adventurous the model is. 0 is always the safest word. We use 0.2, so answers stay steady and factual.
Prompt	The full text we send to the model: the rules, the code, and your question.
Hallucination	When a model confidently says something that is not true. The main danger of AI, and the reason for RAG.

4.2 Why a model alone is not enough: RAG

The model has never seen your repository. If you ask 'where is login?', it can only guess. We could retrain it on your code (fine-tuning), but that is slow, expensive, and goes out of date with every commit.

> **RAG = Retrieval-Augmented Generation**
> Retrieval: first search the repository for the few pieces of code that matter. Augmented Generation: put those pieces into the prompt and ask the model to answer from them. It is an open-book exam: the model may look at the notes. The answer is always based on current code, and we can show which files were used.



4.3 Embeddings: turning meaning into numbers

A computer cannot compare meanings, but it can compare numbers. An embedding model reads a piece of text and outputs a list of numbers that represents its meaning. Texts with similar meaning get similar numbers. Ours is nomic-embed-text, and it outputs 768 numbers for every piece of text. That list of 768 numbers is called a vector.

> **Picture it as a map**
> Imagine a huge map where every piece of text is a dot. 'login' and 'authentication' land close together. 'banana' lands far away. A question is also turned into a dot, and we simply look for the dots nearest to it. The real map has 768 directions instead of 2, but the idea is the same.

A tiny worked example (3 numbers instead of 768)

```
login = (0.90, 0.10, 0.00)
authentication = (0.85, 0.20, 0.05)
banana = (0.00, 0.10, 0.95)

similarity(login, authentication) = 0.99 <- very similar
similarity(login, banana) = 0.01 <- unrelated
```

The measure used is cosine similarity: the angle between two vectors. Same direction means similar meaning (score near 1). Unrelated means score near 0. In the database we compute it as 1 minus the cosine distance (the pgvector operator is <=>). We ignore matches below 0.3 because they are mostly noise.

> **Why embeddings beat keyword search**
> A keyword search for 'authentication' misses a file that only says 'login' and 'session'. An embedding search finds it, because the meaning is close. The weakness: embeddings blur exact names, which is why we added hybrid search in 4.7.

4.4 Where the vectors are stored

- PostgreSQL is a normal database. pgvector is an add-on that gives it a vector column type and fast 'nearest neighbour' search. We use it, so one database holds everything and every search can be limited to one project.
- Qdrant is a database made only for vectors. It is installed and running, but today pgvector does the work. Moving to Qdrant is future scope for very large repositories.
- Every search always includes the project id, so one repository can never read another repository's code. This is tested.

4.5 Chunking: cutting code into the right-sized pieces

We cannot give the model a whole repository: the context window is tiny. We also should not embed a whole file as one dot, because a 500-line file is about many things. So we cut each file into chunks, then embed each chunk.

- We cut at function and class boundaries when possible, so a chunk is usually one complete function. Chunk size is about 1,500 characters with 150 characters of overlap, so nothing important is cut in half.
- Every chunk remembers its file, its line numbers and the function name it starts with. That is how answers can say 'src/lib/github.ts, lines 41 to 60'.
- Before embedding, each chunk gets a label such as 'File: src/lib/ai.ts, function detectBreakingChanges, lines 10-40', so the vector knows where the code lives.
- Why function-level and not whole-file? Precision. A question about one function now finds that function, not a long file that merely mentions it.

4.6 Indexing: what happens when you connect a repository

1. **Download and filter**
   Fetch the files from GitHub, skipping junk such as node_modules, lock files, images and build output.
2. **Choose the important files**
   A summary costs one model call per file, so only the most important files (up to 30 by default) get one. Chunks are cheap, so up to 300 files get chunks.
3. **Summarise**
   The local model writes a short plain-English summary of each important file.
4. **Embed**
   nomic-embed-text turns each summary, and each code chunk, into a 768-number vector.
5. **Store**
   Files, summaries, chunks and vectors go into PostgreSQL. The live progress screen is reading these steps while they happen.
6. **Analyse commits**
   The latest commits are fetched and checked for breaking changes (part 5).

To try CodeMind without GitHub, we added a command: npm run index:local. It indexes a folder on your computer with no internet and no AI summary (a rule-based one-line summary is used instead). It indexed this whole repository, 220 files and 792 chunks, in about 22 seconds.

4.7 Hybrid search: meaning plus exact words

Vector search is good at meaning but can blur exact names like verifyGithubSignature. Keyword search is the opposite. Hybrid search runs both and merges the two ranked lists.

- Vector list: the chunks whose meaning is closest to the question.
- Keyword list: PostgreSQL full-text search for the important words in the question (filler words like 'how' and 'does' are removed).
- Merge with Reciprocal Rank Fusion (RRF). Each chunk scores 1 divided by (60 + its rank) in every list it appears in, and the scores are added. A chunk that both lists like rises to the top.

RRF worked example

```
Vector list : x (rank 1), y (rank 2), z (rank 3)
Keyword list: y (rank 1), w (rank 2), x (rank 3)

score(y) = 1/62 + 1/61 = 0.03252 <- first
score(x) = 1/61 + 1/63 = 0.03226 <- second
score(w) = 1/62 = 0.01613
score(z) = 1/63 = 0.01587
```

Why 60? It is the standard value from the research paper. It stops one list from dominating. The big advantage: RRF uses only ranks, so we never have to compare a vector score with a keyword score, which are on different scales.

> **Honest note**
> Hybrid search is switched off by default (RETRIEVAL_MODE=vector). Switch it on with RETRIEVAL_MODE=hybrid. In our test it helped (part 10), but ten questions is a small test.

4.8 Building the prompt and the guardrails

After search, we have the best chunks. The prompt has three parts:

1. Rules (the system message): use only the code given; say 'No, X is not part of this project' when the code shows something is missing; name the files you used; never answer from general knowledge; if the question is not about this repository, reply with the fixed message 'I could not find enough relevant information'.
2. The context: the chosen chunks, each labelled with its file name and a short summary, cut to fit a budget.
3. The question (and, for a follow-up, the last few questions and answers).

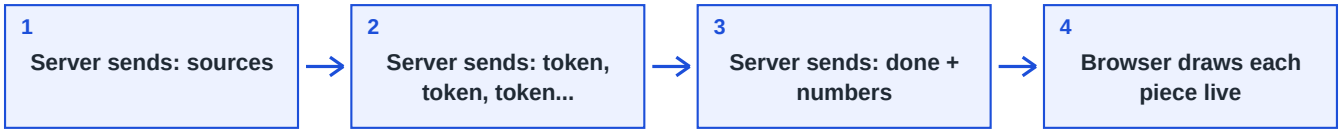
Model	Code given per file	Total code	Chunks searched
Local (Ollama)	1,800 characters	7,500 characters	6
Groq	6,000 characters	14,000 characters	8
Gemini	12,000 characters	48,000 characters	16

Bigger models have bigger windows, so they are given more code. This is called a context budget. The 'grounded' badge on an answer is green only when sources were found AND the model did not decline.

> **The hard truth about small models**
> A 3-billion-parameter model sometimes refuses a question it could answer, or answers an off-topic question from memory. We improved the prompt (version 5) so it now answers 'does it have X?' correctly and refuses trivia, but it is not perfect. That is the trade-off for privacy, and the reason Auto-scale exists.

4.9 Generating and streaming the answer

The model writes one token at a time. Instead of waiting for the whole answer, the server sends each piece to the browser the moment it exists. This is called streaming.



- The data travels as NDJSON: one small JSON message per line, for example {"type":"token","text":"The"}.

- Three message types: sources (shown first), token (a piece of the answer), done (final text plus timings and token counts). A fourth, error, reports problems.
- Stop button: the browser cancels the request. The server notices the connection closed and stops the model too, so no computing power is wasted.
- Smooth drawing: when a fast model sends hundreds of tokens per second, we merge the tokens that arrive between two screen refreshes into one update.

5. The features, one by one

For each feature: what the user sees, how it works inside, and why it matters.

5.1 Ask AI with sources

- User sees: a question box, four modes (With RAG, Without RAG, Auto-scale, Compare both), and answers that list their sources with a match percentage.
- Inside: the pipeline of part 4. 'Without RAG' skips the search, so you can show how much worse an answer is without the code. 'Compare both' shows the two side by side.
- Why it matters: it proves RAG helps. Ask the same question both ways in the demo.

5.2 Auto-scale: a small model first, a big one only if you agree

1. **Answer with the small local model**
   Fast, private, free. Nothing leaves the computer.
2. **Check if the answer looks weak**
   Two simple checks: the model said it could not find enough information, or the answer did not mention any of the files it was given.
3. **Offer a button, never do it silently**
   If weak, CodeMind shows 'Scale up to Gemini or Groq' and says plainly that the code excerpts will be sent to that provider.
4. **Answer again with the bigger model**
   Only after you press the button. The same retrieved code is reused, so only the thinking changes.

5.3 Streaming answers (new)

- User sees: sources first, then words appear one by one with a blinking cursor, a live timer, and a Stop button.
- Inside: each model provider got a stream method. Ollama sends one JSON object per line. Gemini and Groq use Server-Sent Events (lines starting with 'data:'). A shared reader joins lines even when the network cuts them in the middle.
- Why it matters: a local model can take 5 to 10 seconds. Watching the answer arrive feels instant, and people can stop a bad answer early.

5.4 Chat history and follow-ups (new)

- User sees: a chat window. After the first answer, a box for follow-ups, suggestion chips ('Explain in simpler terms'), and a New chat button.
- Problem: 'and where is it used?' means nothing alone. 'It' is in the previous message.

- Fix, two places. (1) Search: if a question is short (5 words or fewer) or contains words like 'it', 'this', 'that', we search with the previous question added. (2) Prompt: the last 3 questions and answers are written into the prompt, with answers shortened to 600 characters so they never crowd out the code.
- Saved in the browser (localStorage) per repository, up to 20 questions, with code excerpts shortened so it never fills storage.

5.5 Compare models (new)

- User sees: pick two to four models (default: local qwen, Gemini, Groq), type one question, press Compare. The answers stream side by side.
- Numbers per model: first word time, total time, speed in tokens per second, answer size in words, the model's size (gigabytes on disk for local; parameter count read from the name for cloud, for example 'gpt-oss-20b' means about 20 billion), and the tokens used.
- A trophy marks the best in each category. Charts and a table summarise at the bottom.
- Inside: cloud models run at the same time. Local models take turns, because one small graphics card runs one model at a time.
- Fairness note shown on screen: each model is given as much code as its window allows, so a bigger window may read more code. That is part of what is being compared.
- Privacy: a warning appears when a cloud model is selected, because code excerpts are sent to it.

5.6 Retrieval lab (new)

Question: how do we know the search is good? Answer: we measure it. You give the lab questions and the file that should be found for each (the answer key). It runs vector search and hybrid search and scores both.

Score	In plain words	Example
Hit rate	Out of all questions, how often did the right file show up in the top results?	8 of 10 found = 80%
MRR	Mean Reciprocal Rank. Rewards the right file being near the top. Rank 1 = 1.0, rank 2 = 0.5, rank 4 = 0.25, not found = 0.	ranks 1, 2, missed = (1 + 0.5 + 0) / 3 = 0.5
Precision	Of the files returned, how many were right? It is low when one right file is returned among six.	1 right of 6 = 17%

- Question sources: the 10 CodeMind sample questions, questions you type, or 'Generate from this repository' (it writes questions from real function names and file descriptions).
- Command-line twin: npm run eval -- <projectId>. The page and the command run the same code, so their numbers always agree.

5.7 Explain this file (new)

- User sees: on Repository Files, an 'Explain this file' button opens a side panel. First a list of the functions and classes with their line numbers appears instantly. Then a plain-English explanation streams in: what the file is for, how it works, key functions, how it fits in.
- The function list is not AI. It is found by scanning the file for declarations, so it is exact. Only the explanation comes from the model: whichever model is chosen in the top-right box (local qwen by default).
- Long files: the model's window is small, so only the start of a very long file is read, and the panel says so ('Only the first N lines were read').

- Explanations are remembered while the page is open, so going back to a file is instant.

5.8 Export report (new)

- User sees: an Export report button, then a small dialog with tick boxes for sections, then a PDF download.
- Sections: overview, languages and tooling, breaking changes, saved answers, AI usage (calls, tokens, speed, by model).
- Inside: the server collects the facts from the database (no internet needed) and draws the PDF itself with the pdf-lib library. Unusual characters such as emoji are replaced, because PDFs built-in fonts cannot draw them.

5.9 Breaking-change detection

1. **Fetch recent commits**
   Up to 20 not-yet-analysed commits are loaded from GitHub.
2. **Read each diff**
   The diff is the list of lines added and removed.
3. **Ask the model for structured JSON**
   Is it breaking? Severity (low, medium, high, critical)? What parts are affected? Is migration needed and how? The model is forced to answer in valid JSON (Ollama's JSON mode).
4. **Store and show**
   Saved on the commit. The dashboard shows an alert, a monitor and charts. It is advisory: it can miss or over-flag changes.

5.10 GitHub webhook: automatic analysis (new)

A webhook is GitHub calling your server when something happens, so you do not have to keep asking. We add one address in the repository settings. When someone pushes to the default branch, GitHub sends a message and CodeMind analyses the new commits by itself.

- Security: GitHub signs every message with a secret (HMAC-SHA256 of the message body). We recompute the signature and compare in constant time. A wrong or missing signature is rejected with HTTP 401. We tested this live.
- Other answers: a ping returns 200. A push to another branch is ignored (202). A push for a repository we do not have is accepted but does nothing.
- CodeMind replies at once and does the slow work in the background, because GitHub only waits a few seconds.

5.11 Metrics: every model call is measured

Each call to any model is saved with its operation (answer, summary, explanation...), model, tokens, time to answer and speed. The AI Metrics page graphs it. Prometheus and Grafana can show live dashboards. This is

how we can say with numbers that Groq writes about 900 tokens per second and our local model about 50.

6. The database: what is stored

PostgreSQL holds nine kinds of record. Prisma is the tool that describes them and talks to the database.

Table	What it holds
`Project`	One connected repository: name, GitHub link, created date, archived flag.
`UserToProject`	Which user belongs to which project. Every read checks this first.
`SourceCodeEmbedding`	One row per indexed file: path, full source code, AI summary, and the summary's vector.
`CodeChunk`	One row per function-sized chunk: file, function name, start and end line, text and its vector.
`Commit`	One analysed commit: message, author, date, AI summary, breaking-change flag, severity, details, migration steps.
`Question`	A saved answer: the question, the answer and the files used.
`LlmCall`	One row per model call: which model, tokens, time, speed, success or error. Powers the Metrics page.
`IndexingJob`	Live progress and final report of one indexing run.
`AppSetting`	Small settings, for example which model is selected.

> **Security idea worth saying out loud**
> There is no sign-in: the app runs as one built-in local user. But every search and every read still checks the project id, so data of one repository can never leak into another. An integration test on a real database proves it.

7. The frontend, explained

7.1 The tools

Tool	What it is, in plain words
React	A library for building screens out of small reusable pieces called components.
Next.js	A framework on top of React. Gives pages, routing and server code in one project.
TypeScript	JavaScript with types, so many mistakes are caught before the program runs.
tRPC	Lets the browser call server functions as if they were local, with the types shared. No hand-written API layer.
Tailwind CSS	Styling by small utility classes written next to the element.
Radix UI and shadcn/ui	Accessible building blocks: dialogs, menus, tabs, tooltips.
Recharts	Draws the bar charts.
Markdown renderer	Turns the model's markdown (headings, lists, code) into formatted text.

7.2 Fluid design: sizes that flow

Most websites change layout at fixed screen widths (breakpoints), so text and spacing jump. We used fluid design: sizes that grow smoothly with the screen.

- Fluid text and spacing use the CSS function clamp(min, preferred, max). For example text-fluid-sm is clamp(0.8rem, 0.76rem + 0.2vw, 0.92rem): never smaller than 0.8rem, never bigger than 0.92rem, and in between it follows the window width. We named these tokens text-fluid-xs to xl and p, gap and spacing fluid-xs to lg.
- Same ratio: charts use aspect-chart (16 by 9) so every chart keeps the same shape on any screen. Model cards use equal-width columns.
- Auto-fit grids: grid-template-columns: repeat(auto-fit, minmax(min(100%, 22rem), 1fr)). The browser fits as many equal columns as there is room for. No breakpoints needed, and the columns also reflow when you open or close the sidebar.
- A subtle bug we found and fixed: the class-merging helper (tailwind-merge) did not know our new names and quietly deleted real colour classes. We taught it the new tokens and added a test.
- Tested at 390 px (phone), 820 px (tablet) and 1440 px (desktop): no sideways scrolling and no console errors.

7.3 Explaining things inside the screens

- Each new page has a 'How this works' box with three numbered steps. It opens the first time and remembers if you close it.
- Small question-mark tooltips explain words like MRR and hit rate where they appear.
- Honest messages: warnings when code will go to the cloud, notices when the index is file-level only, and empty states that say what to do next.

7.4 How the browser handles a streamed answer

1. **Send the question**
   A POST request to /api/ask with the question, project, mode and recent history.

2. **Read the stream**
   The response body is read line by line. Each line is one JSON event.

3. **Update the screen**
   Events are folded into the answer on screen: sources, then text, then final details. Tokens are batched per screen refresh.

4. **Save the chat**
   When nothing is being written, the conversation is saved in the browser (never half-written answers).

8. Quality, DevOps and security

8.1 Testing

- 227 unit tests (28 files) run in under a second. The AI is replaced by a fake, so tests are fast and never depend on a model being installed.
- 8 integration tests run against a real PostgreSQL with pgvector. They prove, for example, that one project can never see another project's code.
- Things tested this time: streaming from all three providers with a fake network, line reading when the network splits a message, follow-up memory, RRF, evaluation scores, webhook signatures, the PDF builder with awkward text, and the class-merging fix.
- Beyond tests: we drove the real app in a real browser, asked real questions to the real local model, and checked the screens at three widths.

8.2 DevOps: automation around the code

Piece	What it does
Docker	Packs the app and everything it needs into a container that runs the same everywhere. Our image is multi-stage (small) and runs as a normal user, not root.
Docker Compose	Starts the app, PostgreSQL and Qdrant with one command.
GitHub Actions: ci.yml	On every push: lint, type-check, unit tests, create the database tables, integration tests on a real pgvector database, production build, Docker build.
GitHub Actions: cd.yml	After CI passes on main, publishes the Docker image to GitHub Container Registry (GHCR).
GitHub Actions: ai-review.yml	On every pull request, runs a tiny local model (qwen2.5-coder:0.5b) inside the runner and posts a summary and a breaking-change verdict as a comment.
Sweep.dev (configured)	An AI agent that turns a GitHub issue starting with 'Sweep:' into a draft pull request. We added sweep.yaml and an issue template. It starts working once its GitHub app is installed.
Qodo PR-Agent (configured)	Reviews pull requests and suggests tests. Added as a workflow. It runs when a GEMINI_API_KEY secret is set, and skips itself otherwise.
Prometheus and Grafana	Collect and draw live numbers such as latency and token use.

> **Be precise about Sweep and Qodo**
> They are set up in the repository, but we have not run them: that needs an account or a key. Say 'configured and ready', not 'working'. Screenshot one real run before you claim more.

8.3 Security and privacy

- Local-first: by default no code leaves the computer.
- Cloud only with consent: choosing a cloud model shows a warning, and Auto-scale asks before sending anything.
- Every search is filtered by project id.
- Input is checked with Zod schemas; GitHub links are parsed strictly; repository code is never executed.
- Webhooks are authenticated with an HMAC signature, compared in constant time.

- Secrets live in the .env file, which git ignores. Important: a GitHub token appeared in a chat during this work, so rotate it on GitHub before the presentation.
- The Docker image runs as a non-root user.

9. What is new, and what was there before

Three layers of work, oldest first. Adjust the 'before' list to match exactly what you presented in the first evaluation.

Layer 0: the starting point (not our work)

git-inspect by Parth Pankaj Singh (MIT licence): the Next.js and tRPC base, GitHub loading, commit analysis and the dashboard, powered by Google Gemini. Always credit it.

Layer 1: built before this week

What	In plain words
Local AI with Ollama	Replaced Gemini as the default. Models sit behind LLMProvider and EmbeddingProvider interfaces, so any model can be swapped in.
Model evaluation	Compared qwen3:4b, qwen3:1.7b and qwen2.5-coder:3b on a 4 GB graphics card. Chose qwen2.5-coder:3b-instruct: fast, direct, fits in memory. The Qwen3 models wasted their words on hidden reasoning.
LangChain RAG pipeline	Retrieval separated from generation, a context budget for the small window, honest 'not enough information' answers, relevance scores, and a no-retrieval baseline.
Cloud models (optional)	Gemini and Groq with a model switcher, live model lists, and clear local or cloud labels.
RAG on / off / compare	Answer with retrieval, without, or both side by side, to show what retrieval adds.
Auto-scale with approval	Small model first, bigger one only when you press a button.
Function-level index	Chunks with line numbers, so answers cite exact functions.
Connect screen	File preview before indexing and live server-side progress with timings.
Repository insights	Languages, Docker, CI, tests and suggestions detected from file names.
Breaking-change monitor	Commit analysis with severity, charts and alerts.
AI Metrics and System Status	Every model call measured; live health of every service; Prometheus and Grafana.
Single local user	Removed the third-party sign-in. Every search stays scoped to one repository.
Tests, Docker, CI/CD	Unit and integration tests, multi-stage Docker image, Compose, GitHub Actions CI, CD to GHCR and an AI pull-request review.
Bug fixes found by tests	Diff summaries counted headers as changes; GitHub links ending with a slash or .git failed to parse; invalid JSON from the model; sign-in redirect.

Layer 2: the engine work for the second evaluation

What	In plain words
Hybrid search	Vector search plus keyword search, merged with RRF. Switch: RETRIEVAL_MODE=hybrid.
Evaluation lab	Scores search quality with hit rate, MRR and precision. Command: npm run eval.
GitHub webhook	A signed push message makes CodeMind analyse new commits automatically.

What	In plain words
Sweep.dev and Qodo set-up	sweep.yaml, an issue template, a Qodo pull-request workflow, and a test-generation guide.
Token streaming in every provider	Ollama, Gemini and Groq can all stream, through one common method.
Follow-up memory	History helpers: trim, detect follow-ups, widen the search, add earlier turns to the prompt.
Symbol scanner	Finds every function, class and method in a file with its line number.
Offline indexer	npm run index:local indexes a folder with no GitHub and no AI summaries.
Prompt version 5	Answers 'does it have X?' correctly and refuses off-topic trivia. 'Grounded' is no longer shown when the model declined.
Tests	From 112 to 227 unit tests, and from 4 to 8 integration tests.

Layer 3: the screens and features for the second evaluation

What	Where to find it
Streaming answers + Stop	Dashboard, Ask box: open the conversation window.
Chat history and follow-ups	Same window: the box at the bottom, 'Conversation (N)' and 'New chat' buttons.
Compare models	Sidebar: Compare Models.
Retrieval lab	Sidebar: Retrieval Lab.
Explain this file	Sidebar: Repository Files, pick a file, press the button.
Export report (PDF)	Dashboard, top right: Export report.
Fluid design	All new screens: smooth sizes, fixed chart shape, auto-fit columns.
Built-in explanations	'How this works' boxes and question-mark tooltips on every new page.

10. Numbers, limits and future scope

Numbers worth remembering (all measured)

Fact	Value
Local answer model	qwen2.5-coder:3b-instruct, 1.9 GB
Embedding model and vector length	nomic-embed-text, 274 MB, 768 numbers
Chunk size and overlap	1,500 and 150 characters
Local context window	3,072 tokens, about 2,000 words
Indexing this repository offline	220 files, 792 chunks, about 22 seconds
Retrieval time per question	about 20 ms (vector), about 50 ms (hybrid)
Writing speed: local / Gemini / Groq	about 48 / 170 / 920 tokens per second
Time to first word: local / Gemini / Groq	1.5 s / 1.1 s / 0.7 s (one test)
Search quality, 10 questions, top 6	Vector: hit rate 80%, MRR 0.65. Hybrid: hit rate 90%, MRR 0.72
Tests	227 unit + 8 integration, all passing

> **How to say the search result honestly**
> 'On our ten test questions, hybrid search found the right file 90 percent of the time instead of 80 percent, and ranked it higher. Ten questions is a small test, so we call it evidence, not proof, and the lab lets anyone test more.'

Honest limits (say these before you are asked)

- The 3-billion-parameter local model is fast and private, but can miss details across many files and may answer some off-topic questions. That is why Auto-scale and Compare models exist.
- Hybrid search is off by default until measured on your own repository. The evaluation is small (10 questions) and written by us.
- Function boundaries come from a text splitter and pattern matching, not a full syntax tree.
- Breaking-change detection is advisory. It can miss or over-flag changes.
- The Explain panel reads only the start of very long files.
- Sweep.dev and Qodo are configured but not yet run.
- Indexing runs while you wait and takes minutes on a laptop for a big repository.

Future scope (each builds on something that already exists)

Idea	Why, and what it builds on
Tree-sitter chunking	Parse the real syntax tree for exact function and class boundaries in every language. Builds on the symbol scanner.
Code graph	Index who calls whom, so 'what breaks if I change this?' can be answered. Grows from the symbol scanner and semantic navigation.

Idea	Why, and what it builds on
Reranking	A second model re-orders the merged results for even better precision. Builds on hybrid search.
Incremental indexing	Re-embed only changed files after a push. The webhook already knows what changed.
Qdrant as the main store	Faster at very large scale. Already installed.
Background jobs and queues	Index without making the user wait.
Accounts and private repositories	GitHub sign-in and several users. Today it is one local user.
Pull-request bot with inline comments	Post the breaking-change analysis on the exact lines. Builds on the AI review workflow.
VS Code extension	Ask questions without leaving the editor.
Bigger local models and a bigger evaluation	Re-run the Retrieval lab on a 7B to 14B model and on more repositories.

11. Presenting it tomorrow

Before you start (checklist)

1. Open Docker Desktop. Wait until it says running.
2. In the project folder run: docker compose up -d db qdrant
3. Make sure Ollama is running (the llama icon in the menu bar). Both models are already downloaded.
4. Run: npm run dev and open http://localhost:3000. Pick your repository in the sidebar.
5. Do one test question first so the model is warm. The first answer after a restart is slower.
6. Rotate the GitHub token that appeared in chat, and keep your API keys private.
7. Take screenshots of every screen as a backup in case something fails live.

A five-minute demo

Minutes	What you do	What you say
0:00	Show the dashboard.	The problem: reading code is slow. CodeMind has read this repository and everything runs on my laptop.
0:45	Ask a question. Let the answer stream. Point at the sources.	Sources come first, so you can check the proof. This is RAG: look up, then answer.
1:45	Ask a follow-up: 'and where is it used?'	It remembers the last three questions. Show the 'Follow-up' badge.
2:15	Try 'Compare both' once (with and without RAG).	Without the code the model only guesses. That is what retrieval adds.
2:45	Open Compare Models. Run three models.	Local is private and free. Cloud is faster. Here are the numbers, measured.
3:45	Open Retrieval Lab. Run the sample.	We measure search quality. Hybrid improved hit rate from 80 to 90 percent on ten questions.
4:20	Explain this file, then Export report.	Plain-English explanation, and a PDF anyone can read.
4:50	Close with the future scope slide.	Tree-sitter, a code graph, reranking and a VS Code extension.

Questions a teacher may ask, with answers

Q. What problem does it solve?
It cuts the time to understand an unfamiliar codebase, by answering questions about it with proof from the code.

Q. Why run the AI locally?
Privacy: the code never leaves the machine. Also no cost and no API key. The price is a smaller, less clever model.

Q. Why RAG and not training the model on the code?
Training (fine-tuning) is slow, costly and out of date after the next commit. RAG always uses the current code and can show its sources.

Q. What is an embedding?
A list of 768 numbers that captures the meaning of a piece of text. Similar meaning gives similar numbers, so we can search by meaning.

Q. Why pgvector and not a separate vector database?
One database for everything, simple to run, and every search can be limited to one project. Qdrant is ready if we outgrow it.

Q. Why split code into function-sized chunks?
The model's window is small and a whole file is about many things. Chunks are precise and let us cite line numbers.

Q. What is hybrid search and why?
Meaning search plus exact-word search, merged with Reciprocal Rank Fusion. Exact names like verifyGithubSignature are caught by keywords.

Q. How do you know it works?
The Retrieval lab scores it: hit rate, MRR and precision. On ten questions hybrid improved hit rate from 80 to 90 percent. It is a small test, and we say so.

Q. How do you stop it making things up?
Rules in the prompt, a fixed 'not enough information' reply, sources shown for every answer, and a 'grounded' badge that is only green when real code was used.

Q. It still answered an off-topic question. Why?
A 3-billion-parameter model follows rules less strictly than a big one. Prompt version 5 improved it; Auto-scale and larger models improve it further.

Q. How does streaming work?
The server sends one JSON message per line as the model writes. The browser reads the lines and draws the text. Stop cancels the request and the model.

Q. How does it remember the conversation?
The last three questions are added to the search and to the prompt. The chat is saved in the browser.

Q. What did you build, and what did you reuse?
The base is git-inspect (MIT licence, credited). We added local AI, RAG, function-level search, hybrid search, evaluation, streaming, chat, compare, explain, report, automation and tests.

Q. How is it tested?
227 fast unit tests with a fake AI, 8 integration tests on a real database, and manual tests in a real browser at three screen sizes.

Q. What is CI/CD?
Automation on GitHub: every push is linted, tested and built, and a Docker image is published. Continuous Integration checks, Continuous Delivery ships.

Q. What are Sweep and Qodo here?
Sweep turns an issue into a draft pull request. Qodo reviews pull requests and suggests tests. Both are configured; say whether you have run them.

Q. What is a webhook, and is it safe?
GitHub calls our server on every push. We verify an HMAC-SHA256 signature using a shared secret, so fake requests are rejected.

Q. What are the limits?
Small local model, small evaluation, advisory breaking-change checks, long files only partly read.

Q. What would you do next?
Tree-sitter chunking, a code graph, reranking, incremental indexing, accounts, and a VS Code extension.

12. Glossary: every word in simple language

Alphabetical. The last column tells you where CodeMind uses it.

Term	Plain meaning	Used in
API	A set of web addresses a program offers so other programs can talk to it.	/api/ask, /api/report
Aspect ratio	The shape of a box, width divided by height. 16 by 9 is a wide screen.	Charts
Auto-scale	Answer with the small model first, offer a bigger one if the answer looks weak.	Ask AI
Breaking change	A code change that can make other code stop working.	Commit analysis
Chunk	A small piece of a file, usually one function.	Search index
CI/CD	Continuous Integration and Delivery: automatic checks and releases on every change.	GitHub Actions
clamp()	A CSS function that keeps a size between a minimum and a maximum while following the screen width.	Fluid design
Commit	One saved change to the code, with a message and an author.	Dashboard
Context window	How much text a model can read at once, counted in tokens.	Prompt budget
Cosine similarity	A score from 0 to 1 for how alike two vectors are, by the angle between them.	Vector search
Diff	The lines added and removed in a commit.	Breaking changes
Docker	A way to pack an app and what it needs into one portable container.	Deployment
Embedding	A list of numbers that stands for the meaning of a text.	Search
Evaluation	Measuring quality with numbers on questions with known answers.	Retrieval lab
Fine-tuning	Retraining a model on new data. Slow and costly. We use RAG instead.	Why RAG
Fluid design	Sizes that grow smoothly with the screen instead of jumping at breakpoints.	All new screens
GHCR	GitHub Container Registry, a place that stores Docker images.	CD workflow
Grounded	An answer that is based on code that was actually found.	Answer badge
Hallucination	A confident answer that is not true.	Why guardrails
HMAC	A signature made from a message and a shared secret, to prove who sent it.	Webhook
Hit rate	Share of questions where the right file was found in the top results.	Retrieval lab
Hybrid search	Meaning search plus exact-word search, merged.	Retrieval
Indexing	Reading a repository and storing it in a searchable form.	Connect screen
JSON	A simple text format for data: names and values in curly brackets.	Model output, streaming
LangChain	A library for building AI apps from standard parts: prompts, models, parsers.	RAG chain
LLM	Large Language Model: a program that writes text by predicting the next word.	qwen2.5-coder, Gemini
localStorage	A small storage area inside the browser.	Chat history

Term	Plain meaning	Used in
MRR	Mean Reciprocal Rank: average of 1 divided by the rank of the right file.	Retrieval lab
NDJSON	One JSON message per line. Easy to send as a stream.	Streaming
Ollama	A program that runs open AI models on your own computer.	Local AI
Parameter	One learned number inside a model. 3B means 3 billion.	Model size
pgvector	A PostgreSQL add-on for storing and searching vectors.	Memory layer
PostgreSQL	A popular open-source database.	All data
Precision	Of the files returned, the share that were right.	Retrieval lab
Prisma	A tool that describes tables in code and gives safe database access.	Database
Prompt	The full text sent to a model: rules, code and question.	Answering
Pull request	A proposal to merge changes into the main code, reviewed first.	CI, Sweep, Qodo
Qdrant	A database built only for vectors.	Future store
RAG	Retrieval-Augmented Generation: search first, then answer from what was found.	Core idea
Reranking	Re-ordering search results with a second, smarter model.	Future scope
Repository	A project folder tracked by Git, usually on GitHub.	Everything
RRF	Reciprocal Rank Fusion: merge ranked lists by adding 1 divided by (60 + rank).	Hybrid search
Server-Sent Events	A way for a server to push a stream of text lines to a browser or program.	Cloud streaming
Streaming	Sending a result piece by piece as it is made.	Answers
Temperature	How adventurous a model is. Low is steady.	0.2 here
Token	A piece of a word, about four characters.	Limits and speed
tRPC	Calls server functions from the browser with shared types.	Frontend to server
Vector	A list of numbers. Here, 768 of them.	Embeddings
Vector database	A database that can find the nearest vectors quickly.	pgvector, Qdrant
Webhook	A call from one service to another when something happens.	GitHub push
Zod	A library that checks that data has the right shape.	Input validation